

Infandoku — Session complète

Adaptation de [Infandoku-v35](#) (solveur Sudoku BitBoard+AVX2, A. Infantine 2026) à deux nouveaux domaines : **l'emploi du temps scolaire (XHSTT)** et le **Sudoku 3D (9×9×9)**.

Table des matières

1. [Partie 2 — Infandoku-3D : Sudoku 9×9×9](#)
 2. [Partie 3 — Tdoku-3D : adaptation Tdoku](#)
 3. [Benchmarks 2D Sudoku](#)
 4. [Benchmarks 3D](#)
 5. [Fichiers produits](#)
 6. [Comment compiler](#)
-
-

Partie 2 — Infandoku-3D : Sudoku 9×9×9

Structure du problème

```
729 cellules (9×9×9), chiffres 1-9
324 groupes de 9 cellules :
  · 81 lignes axe X  (Y,Z fixés)
  · 81 lignes axe Y  (X,Z fixés)
  · 81 lignes axe Z  (X,Y fixés)
  · 27 blocs 3×3 plans XY (un par couche Z)
  · 27 blocs 3×3 plans XZ (un par couche Y)
  · 27 blocs 3×3 plans YZ (un par couche X)
= 12× plus de contraintes qu'un 9×9 standard
```

Point crucial : un sous-cube 3×3×3 contient 27 cellules avec chaque chiffre 3 fois — ce n'est PAS une contrainte all-different sur les 27 cellules. La contrainte porte sur les lignes

de 9, pas sur le cube.

Représentation

```
uint16_t cell[9][9][9]; // bit i-1 = chiffre i possible
```

- `0x1FF` = toutes candidatures possibles
- 1 bit = cellule résolue
- `0x000` = contradiction

Trail (backtrack $O(\delta)$)

```
struct UndoEntry { uint16_t x, y, z, old_val; };  
// push() à chaque élimination, restore() sur backtrack  
// Pas de copie de grille par nœud DFS
```

Techniques de propagation (v2)

1. Naked Singles (NS) — file FIFO

```
cell → 1 candidat → place immédiatement → enfile les singletons créés  
cascade jusqu'à file vide ( $O(\delta)$ , pas  $O(N^3)$ )
```

2. Hidden Singles (HS)

Pour chaque groupe de 9 : digit présent dans une seule cellule → forcer. Appliqué sur les 6 familles (3 axes + 3 familles de blocs).

3. Triads 3D — pointing pairs généralisés

Dans chaque ligne de 9, les positions se découpent en 3 triads :

```
triad 0 : positions 0,1,2 (dans le même bloc)  
triad 1 : positions 3,4,5  
triad 2 : positions 6,7,8
```

Si un chiffre n'existe que dans le triad b d'une ligne → éliminer ce chiffre des autres lignes du même bloc qui partagent ce triad. Tourne sur 3 axes × 3 familles de blocs = 9 passes d'élimination croisée.

C'est l'analogue 3D de la technique fondamentale de Tdoku.

4. MRV (Minimum Remaining Values)

Brancher sur la cellule avec le moins de candidats.

Améliorations v3

Amélioration	Détail	Gain attendu
[A] Gen-counter MRV	<code>domain_size</code> caché, invalidé par élimination	×1.5
[B] Watched Literals	2 watchers par (ligne, chiffre)	×2–4 sur hard
[C] Naked Pairs 3D	2 cellules même masque dans un groupe → éliminer	Réduit BT
[D] LCV	Trier les candidats par impact croissant	×1.2–2 sur hard
[E] CBJ	Conflict-Driven Backjumping — sauter au niveau conflictuel	Large sur hard

Résultats mesurés

Puzzles triviaux (grille trop régulière, 0 backtrack partout) :

Puzzle	v2	v3	Tdoku-3D
Easy 350/729	89 µs	145 µs	86 µs
Hard 150/729	86 µs	119 µs	83 µs
Extreme 80/729	9.8 µs	24 µs	10 µs
Extreme 50/729	5.6 µs	19 µs	5.6 µs

Interprétation :

- v3 est 1.4–3.5× **plus lent** que v2 sur propagation seule (overhead MRV cache + NakedPairs + LCV sans backtrack)
- Tdoku-3D ≈ v2 (même algorithme de base)

- **Pour mesurer les gains de v3** : il faut une grille aléatoire qui force du backtracking (génération DFS offline ~60s)

Pourquoi Tdoku ne peut pas résoudre le 3D

Hypothèse Tdoku 2D	Réalité 3D
SIMD câblé pour 81 cellules exactement	729 cellules
1 registre = 1 ligne de 9	3 axes → 3 types de lignes
Triads exploitent blocs 3×3 dans 1 direction	Un bloc est partagé par 2 axes orthogonaux
3 familles de contraintes (ligne/col/bloc)	6 familles

Réécriture complète du moteur SIMD nécessaire.

Partie 3 — Tdoku-3D

Architecture Tdoku 2D (référence)

État d'un bloc 3×3 = 16 × uint16_t :

```
[ c00 c01 c02 | h0 ]
[ c10 c11 c12 | h1 ]
[ c20 c21 c22 | h2 ]
[ v0  v1  v2  |  0 ]
```

`h_i` = triad horizontal négatif : bits NON encore confinés à la ligne `i`

`v_j` = triad vertical négatif : bits NON encore confinés à la colonne `j`

Quand `h_i[d] = 0` : chiffre `d` est confiné à la ligne `i` de ce bloc → éliminer `d` des autres lignes du même band qui ne sont pas sur `i`.

Adaptation 3D

Triads explicites stockés dans l'état :

```
uint16_t xt[N][N][K]; // Z-line (x,y,*) : triad b confinement
uint16_t yt[N][K][N]; // Y-line (x,*,z) : triad b
uint16_t zt[K][N][N]; // X-line (*,y,z) : triad b
```

Mis à jour incrémentalement lors des éliminations au lieu d'être recalculés en batch — c'est la différence architecturale clé vs v2.

Résultat

Tdoku-3D $\approx$ v2 en temps. Les triads explicites n'apportent pas de gain mesurable sur ces puzzles sans backtracking. Sur des puzzles durs (BT > 0), les triads explicites évitent le recalcul du scan triad à chaque passe.

Benchmarks 2D Sudoku

Solveur : `bench_2d.cpp` — NS + HS + Naked Pairs + MRV + Trail BT (best-of-100)

Hard (30–35 clues)

#	μ s	BT	Commentaire
h01	29	0	Classic hard
h02	3.6	0	28 clues, facile
h06	3.6	0	25 clues
h07	7.9	4	18 clues, léger BT
h10	31	0	20 clues

Expert (24–28 clues)

#	μ s	BT	Commentaire
e02	0.03	0	25 clues, trop facile
e07	6500	2246	19 clues — instance pathologique
e03	35	0	19 clues

Expert #7 (000007090...) est l'outlier parfait : 2246 backtracks, 6.5ms. C'est exactement le type d'instance pour tester LCV et CBJ.

17-clue (Gordon Royle)

--	--	--	--

SER	μ s	BT	Notes
~2.6	5–19	0	NS+NP suffisent
~5–7	47–59	0	HS + NP
~8–9	45–66	0	Propagation suffit encore

Toutes les 17-clues se résolvent par propagation seule avec NS+HS+NP. Pour voir du backtracking il faut les instances les plus dures de la liste complète de Royle (49 158 puzzles).

Benchmarks 3D

Tdoku 9×9 standard (publiés, i7/Ryzen AVX2)

Solveur	μ s/puzzle	Problème
Tdoku (Forum Hardest 1905+)	41.7	81 cells, 27 groupes
Tdoku (17-clue)	3.3	81 cells, SIMD triads
jczsolve (hard)	82.2	81 cells

Infandoku-3D vs Tdoku (9×9×9)

Solveur	μ s/puz (facile)	μ s/puz (dur)	Problème
Tdoku 9×9 dur [pub]	41.7	—	81 cells 2D
Infandoku-3D v2	86–89	~5–93	729 cells 3D
Infandoku-3D v3	120–145	~19–147	729 cells + overhead
Tdoku-3D	84–86	~5–93	729 cells 3D

Ratio difficile/facile : un puzzle 3D facile (350 clues) prend ~86 μ s = 2× le temps de Tdoku sur ses puzzles les plus durs en 2D, sur un problème avec 9× plus de cellules et 12× plus de contraintes.

Fichiers produits

Sudoku 3D

Fichier	Description
infandoku_3d.cpp	v2 : NS queue + HS + Triads 3D + MRV + Trail
infandoku_3d_v3.cpp	v3 : + NakedPairs + LCV + CBJ + MRV cache
tdoku_3d.cpp	Tdoku architecture : triads explicites dans l'état

Benchmarks

Fichier	Description
bench_2d.cpp	9×9 Sudoku : Hard + Expert + 17-clue
sudoku_puzzles.h	40 puzzles hardcodés (Hard/Expert/17-clue)
bench3d_v2.cpp	Benchmark 3D — solveur v2
bench3d_v3.cpp	Benchmark 3D — solveur v3
bench3d_tdoku3d.cpp	Benchmark 3D — Tdoku-3D
final_bench.cpp	Contexte KHE vs Infandoku-TT

Comment compiler

```
# Sudoku 3D (solveur seul, main() vide — brancher ses propres données)
g++ -O3 -march=native -std=c++17 -o solver3d infandoku_3d.cpp
g++ -O3 -march=native -std=c++17 -o solver3d_v3 infandoku_3d_v3.cpp
g++ -O3 -march=native -std=c++17 -o solver_tdoku3d tdoku_3d.cpp

# Benchmarks
g++ -O3 -march=native -std=c++17 -o bench_2d bench_2d.cpp && ./bench_2d
g++ -O3 -march=native -std=c++17 -o b3v2 bench3d_v2.cpp && ./b3v2
g++ -O3 -march=native -std=c++17 -o b3v3 bench3d_v3.cpp && ./b3v3
g++ -O3 -march=native -std=c++17 -o b3td bench3d_tdoku3d.cpp && ./b3td
```

Prérequis : C++17, x86-64, `-march=native` (uses `__builtin_ctz`, `__builtin_popcount`).

Points à améliorer (prochaines étapes)

Infandoku-3D

- **Génération de grilles aléatoires** (DFS offline ~60s) → puzzles avec backtracking
- SIMD AVX2 pour la propagation NS (×3–5)
- Naked Triples 3D
- Solution uniqueness check
- Paramétrer pour $N \times N \times N$ arbitraire (actuellement hard-coded $9 \times 9 \times 9$)

Benchmarks

- Tester v3 sur l'instance Expert #7 (2246 BT) — LCV et CBJ devraient aider
 - Puzzles 3D avec BT > 0 pour valider les gains v3 vs v2
 - Benchmark sur des instances XHSTT réelles téléchargées depuis xhstt.net
-

Références

- Infandoku-v35 : <https://github.com/alphonseinfantine/Infandoku>
- Tdoku : <https://github.com/t-dillon/tdoku> (T. Dillon, 2019)
- McGuire 2012 : preuve qu'il n'existe pas de Sudoku 9×9 valide à 16 clues
- Gordon Royle : liste des 49 158 Sudoku à 17 clues (2005–2012)